DWH MCP EXPLORER / STAGE 6.9.4

РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ И НАСТРОЙКЕ

Deployment baseline v1.0 · 7 November 2026 · Русская версия

Пошаговая настройка локальной версии DWH MCP Explorer на Windows: PostgreSQL, Azure Databricks, Claude Desktop, проверки работоспособности и безопасность перед публикацией на GitHub.

1. Назначение, границы решения и допущения

Документ описывает подтверждённую локальную конфигурацию для разработки, а не серверный облачный сервис. Claude Desktop запускает Python MCP-сервер локальным процессом; server.py регистрирует 21 инструмент, использующий PostgreSQL / SQL Repository и провайдеры Azure Databricks SDK. Knowledge Object формируется по запросу; постоянной векторной базы знаний пока нет.

Рабочее окружение: Windows, PowerShell, VS Code; Python 3.11 и uv; локальный PostgreSQL; Azure Databricks с Unity Catalog и доступным SQL Warehouse. Идентификаторы аккаунтов, персональные сведения, токены, пароли и закрытые параметры намеренно исключены.

2. Предварительные требования и каталог проекта

Компонент	Требования / примечания
Python	uv; Python 3.11 через uv или локальную установку
Исходный код	server.py, dwh_mcp/, sql_sources/, tests/, pyproject.toml
PostgreSQL	Запущенная локальная СУБД, имя базы и права чтения метаданных
Azure Databricks	Доступ к Workspace, Unity Catalog и запросам через SQL Warehouse
Databricks CLI	Команда databricks доступна из терминала
Claude Desktop	Установленное приложение с поддержкой локальных MCP-коннекторов

3. Подготовка Python-окружения

В терминале PowerShell из корневой папки уже существующего проекта восстановите зависимости из имеющегося lock-файла. Приведённые команды не создают новый проект с нуля.

```
cd C:\path\to\dwh-mcp-explorer
uv sync
uv run python --version
uv run python -c "from server import mcp; print('MCP import OK')"
```

Если uv sync не работает из-за отсутствия pyproject.toml или uv.lock, сначала восстановите эти файлы из репозитория, а не создавайте произвольный набор зависимостей. Папку .venv не нужно добавлять в Git.

4. Настройка PostgreSQL и SQL Repository

Запустите PostgreSQL и убедитесь, что доступны база проекта и учебные объекты DWH. В рамках Stage 3–5 использовались схемы gold и staging, таблицы gold.dim_customer и gold.fact_orders, включая внешний ключ от fact_orders к dim_customer. В SQL Repository использовались файлы dim_customer.sql, fact_orders.sql и негативный тестовый пример.

Точные имена параметров подключения зависят от реализации: откройте dwh_mcp/database.py и способ загрузки конфигурации, чтобы проверить ожидаемые переменные окружения. Следующий блок PowerShell — ТОЛЬКО ПРИМЕР; нельзя утверждать, что сервер читает эти имена без проверки кода.

```
# Illustrative only: verify actual variables in dwh_mcp/database.py
$env:PGHOST = "localhost"
$env:PGPORT = "5432"
$env:PGDATABASE = "<database>"
$env:PGUSER = "<read_only_user>"
# Never paste a real password into a public document.
```

Пароли должны храниться в локальном .env или менеджере секретов, если это поддерживается кодом; альтернативно — в переменных окружения пользователя. Не помещайте пароль в server.py, общедоступные скриншоты и Git. До запуска Claude выполните существующий тест соединения с PostgreSQL.

5. Azure Databricks: CLI и OAuth

Авторизуйтесь через существующий профиль CLI MARKETPLACE_LAB и браузерный OAuth. Конфигурация профилей обычно хранится в %USERPROFILE%\.databrickscfg, а OAuth-данные — отдельно в защищённом хранилище CLI. Кэшированные токены нельзя копировать в репозиторий.

```
databricks auth login --profile MARKETPLACE_LAB
databricks auth describe --profile MARKETPLACE_LAB
# If configured: databricks auth profiles
```

В наших провайдерах подтверждено использование WorkspaceClient(profile="MARKETPLACE_LAB"). Метаданные читаются через Unity Catalog API; Table/Column Lineage — из system.access.table_lineage и system.access.column_lineage; проверка Grain — через SQL Statement Execution с запросом SELECT.

Для другого Workspace проверьте параметры warehouse_id по умолчанию в dwh_mcp/databricks_lineage_provider.py и dwh_mcp/databricks_grain_provider.py. В текущем проекте ID задан в настройке конструктора; он не является переносимым между Workspace. Для GitHub желательно вынести его в конфигурацию.

OAuth-сессия может устареть между рабочими днями. Мы восстанавливали доступ повторным databricks auth login --profile MARKETPLACE_LAB с проверкой auth describe. Ошибка refresh token относится к авторизации и сама по себе не указывает на дефект Python-провайдера.

6. Подключение MCP в Claude Desktop

Локальный сервер настраивается в реальном файле %APPDATA%\Claude\claude_desktop_config.json. Фрагмент ниже — очищенный ШАБЛОН: замените C:\path\to\dwh-mcp-explorer на действительный локальный путь и проверьте синтаксис итогового JSON.

```
{
  "mcpServers": {
    "dwh-metadata-explorer": {
      "command": "C:\\path\\to\\dwh-mcp-explorer\\.venv\\Scripts\\python.exe",
      "args": ["-u", "C:\\path\\to\\dwh-mcp-explorer\\server.py"]
    }
  }
}
```

Укажите Python из .venv проекта; параметр -u включает небуферизованный вывод. Протокол MCP использует stdin/stdout, поэтому произвольные сообщения на stdout могут нарушить обмен. Логи направляйте в stderr или штатную систему логирования.

Перезапустите Claude Desktop, затем в Settings → Customize → Connectors проверьте локальный коннектор dwh-metadata-explorer. В новом чате включите его через меню +. В нашей проверке переключение Tool access на Tools already loaded помогло устранить проблему обнаружения инструмента.

Обнаружение инструмента ещё не подтверждает доступность СУБД. Сначала попросите Claude проверить наличие databricks_investigate_object без вызова; затем при необходимости выполняйте интеграционный сценарий, учитывая стоимость SQL Warehouse.

7. Проверки после развёртывания

Сначала запускайте локальные тесты реестра и модулей с Mock; они не требуют SQL Warehouse. Реальные Databricks-тесты приведены отдельно: они обращаются к облачной среде и могут запустить платные вычислительные ресурсы.

Проверка	Команда	Тип
Импорт сервера	uv run python -c "from server import mcp; print('OK')"	Локально
Реестр	uv run python -m tests.test_stage6_cross_platform_registry	Локально
Knowledge Tool	uv run python -m tests.test_stage6_knowledge_tool	Mock / локально
Grain Tool	uv run python -m tests.test_stage6_grain_tool	Mock / локально
Ошибки MCP	uv run python -m tests.test_stage6_mcp_error_response	Mock / локально
Stage 5 Composition	uv run python -m tests.test_stage5_composition	Локальный MCP
Knowledge Integration	uv run python -m tests.test_stage6_knowledge_integration	РЕАЛЬНЫЙ / платный
Grain Integration	uv run python -m tests.test_stage6_grain_integration	РЕАЛЬНЫЙ / платный

Контрольный снимок marketplace_lab.gold.fact_orders содержал 10 000 строк и 10 000 уникальных order_id, без NULL и избыточных строк-дубликатов. Это зафиксированный результат, а не универсальное требование для будущих загрузок. Lineage возвращал по 50 событий при окне 45 дней и LIMIT 50; это не доказывает полноту графа.

8. Безопасность и проверка перед GitHub

• Не публиковать .env, .venv, .databrickscfg, локальный конфиг Claude Desktop, OAuth-кэши, токены, пароли и приватные абсолютные пути.

• Создать .env.example только с именами переменных и безопасными заглушками. Не помещать реальные секреты в PDF или README.

• Проверить .gitignore, затем git status и git diff --cached перед коммитом. Выполнить поиск секретов и проверить также историю Git, включая удалённые файлы.

• Использовать роли PostgreSQL с минимальными правами и точечные разрешения Unity Catalog / SQL Warehouse. Сам по себе MCP не гарантирует ограничение доступа.

• Проверить SQL-примеры на наличие персональных данных, внутренних хостов, идентификаторов, закрытой бизнес-логики и кода, принадлежащего работодателю.

• Не изменять рабочие таблицы. Даже SELECT может потреблять ресурсы SQL Warehouse.

• До публичного выпуска убрать жёстко заданные локальные пути, проверить warehouse ID в коде и добавить конфигурационный способ его переопределить.

```
git status --short
git diff --cached --name-only
# Review staged files and repository history for secrets.
# Add .env, .venv/, token caches and private config to .gitignore.
```

9. Диагностика типичных проблем

Симптом	Действие
Недействительный OAuth refresh token	Повторить databricks auth login --profile MARKETPLACE_LAB и проверить auth describe.
Claude не видит MCP Tool	Перезапустить приложение; включить коннектор в ЭТОМ чате; проверить Tool access.
Инструмент виден, но вызов падает	Посмотреть stderr/логи сервера, отличить исключение провайдера от MCP is_error.
PostgreSQL / psycopg	Проверить службу БД, Python-окружение и параметры; повторить изолированный тест.
Тест ждёт 17 или 20 инструментов	Убрать устаревший фиксированный счётчик; проверять критические имена.
Lineage показывает ровно 50 событий	Запрос ограничен LIMIT 50; truncated=false не доказывает полной истории.
SQL Warehouse: ошибка или тайм-аут	Проверить Warehouse, права и сессию; CANCEL после 30 секунд — повод диагностировать запрос.

10. Критерии приёмки и следующий этап

Минимальные критерии: зависимости Python установлены; server.py импортируется; реестр содержит 21 инструмент; выбранные локальные регрессионные тесты проходят; Claude Desktop видит активный локальный MCP-коннектор. Доступность PostgreSQL и Databricks проверяется отдельно для каждого окружения.

Это руководство основано на реально проверенном учебном окружении. Точные переменные PostgreSQL, адрес будущего GitHub-репозитория и переносимый способ задания Warehouse ID нужно окончательно сверить с исходным кодом перед публикацией. Stage 6.9.5 завершит отчётность; настройка публичного репозитория, README и демосценарии — отдельная следующая задача.